

Deep Learning — Complete Course Review

Lessons 1 – 25 | Every Key Concept, Function, Method & Code Syntax

This review guide is a detailed lesson-by-lesson reference. It covers every important concept, function, method, argument, activation, loss function, training trick, and code syntax learned throughout the course. Use it as both a study guide and a quick-reference cheat sheet.

■ Lesson 1: NumPy Neural Network from Scratch (FNN on Digits Dataset)

Built a full Feedforward Neural Network (FNN) with 2 hidden layers entirely in NumPy, trained on the sklearn Digits dataset (64 features, 10 classes). Also compared with sklearn's MLPClassifier.

1.1 Dataset & Preprocessing

- **load_digits():** Loads an 8x8 handwritten digit image dataset (1797 samples, 64 features, 10 classes)
- **train_test_split(X, y, test_size=0.2, stratify=y):** Splits data; stratify=y ensures each class is proportionally represented in both sets
- **StandardScaler().fit_transform(X_train):** Normalizes features to zero mean, unit variance. Must .fit() on train, only .transform() on test
- **np.eye(num_classes)[y_train]:** Creates one-hot encoded matrix from integer class labels. e.g. class 3 → [0,0,0,1,0,...]

1.2 Activation Functions (implemented in NumPy)

Function	Formula / Implementation
ReLU	$\text{np.maximum}(0, x)$ — outputs x if positive, 0 otherwise
ReLU Derivative	$(x > 0).astype(float)$ — 1 where $x > 0$, 0 otherwise

Softmax

$\exp(x) / \sum(\exp(x))$ — converts logits to probabilities; numerically stable with $x - \max(x)$

Cross-Entropy Loss

$-\text{mean}(\sum(y_{\text{true}} * \log(y_{\text{pred}})))$ — measures divergence between predicted and true distribution

1.3 He (Kaiming) Weight Initialization

- **Formula:** $W = \text{np.random.randn}(\text{fan_in}, \text{fan_out}) * \text{np.sqrt}(2 / \text{fan_in})$
- **Why?:** Prevents vanishing/exploding gradients in ReLU networks. Factor $\text{sqrt}(2/\text{fan_in})$ keeps variance stable across layers.

```
self.W1 = np.random.randn(input_size, hidden1_size) * np.sqrt(2 / input_size)
self.b1 = np.zeros((1, hidden1_size))
```

1.4 Forward Pass

Each layer: $Z = X @ W + b$ then $A = \text{activation}(Z)$

```
self.Z1 = X @ self.W1 + self.b1    # Linear transform
self.A1 = relu(self.Z1)            # Activation
self.Z3 = self.A2 @ self.W3 + self.b3
self.A3 = softmax(self.Z3)         # Output probabilities
```

1.5 Backpropagation (Manual)

- **Output gradient:** $dZ3 = A3 - y_{\text{true}}$ (derivative of cross-entropy + softmax combined)
- **Weight gradient:** $dW = A_{\text{prev}}.T @ dZ / m$ (m = batch size)
- **Bias gradient:** $db = \text{np.sum}(dZ, \text{axis}=0, \text{keepdims}=\text{True}) / m$
- **Backprop through ReLU:** $dZ_{\text{prev}} = dA * \text{relu_derivative}(Z_{\text{prev}})$
- **Update rule:** $W -= \text{lr} * dW, b -= \text{lr} * db$

1.6 Evaluation Metrics

Metric / Function	Purpose
-------------------	---------

accuracy_score(y_true, y_pred)	Fraction of correctly classified samples
precision_score(..., average='weighted')	TP / (TP+FP) per class, weighted by support
recall_score(..., average='weighted')	TP / (TP+FN) per class, weighted by support
f1_score(..., average='weighted')	Harmonic mean of precision & recall
confusion_matrix(y_true, y_pred)	Matrix of actual vs predicted classes
classification_report(y_true, y_pred)	Full per-class precision, recall, F1, support table
ConfusionMatrixDisplay(cm).plot(cmap='Blues')	Visualizes confusion matrix as a heatmap

1.7 sklearn MLPClassifier

```

from sklearn.neural_network import MLPClassifier

mlp = MLPClassifier(
    hidden_layer_sizes=(128, 64), # tuple → 2 hidden layers
    activation='relu',
    solver='sgd',                 # optimizer: 'sgd', 'adam', 'lbfgs'
    learning_rate_init=0.01,
    max_iter=300,
    random_state=42
)

mlp.fit(X_train, y_train)
mlp_pred = mlp.predict(X_test)

```

■ Lesson 2: Micrograd — Custom Autograd Engine from Scratch

Implemented a scalar automatic differentiation library (like PyTorch's autograd, but minimal). Every operation builds a computational graph; `.backward()` computes gradients via the chain rule.

2.1 The Value Class

```
class Value:
    def __init__(self, data, _children=(), _op="", label=""):
        self.data = float(data)
        self.grad = 0.0          # accumulated gradient
        self._backward = lambda: None # gradient function
        self._prev = set(_children) # parent nodes
        self._op = _op           # operation that created this node
```

2.2 Key Concepts

- **Computational Graph (DAG):** Each operation creates a new Value node. Parents are stored in `_prev`. The graph is a Directed Acyclic Graph (DAG).
- **Chain Rule:** $d(\text{loss})/d(W) = d(\text{loss})/d(\text{out}) * d(\text{out})/d(W)$. Each `_backward` function implements the local gradient and multiplies by `out.grad`.
- **Topological Sort:** Ensures gradients flow from output (loss) to inputs in correct dependency order.
- **self.grad accumulates:** Gradient is `+=` not `=`, because a node can be used by multiple downstream nodes (fanout).

2.3 Operator Implementations

Operator	Forward / Backward Logic
<code>__add__(self, other)</code>	<code>out = self.data + other.data self.grad += out.grad; other.grad += out.grad</code>
<code>__mul__(self, other)</code>	<code>out = self.data * other.data self.grad += other.data * out.grad</code>
<code>__pow__(self, exp)</code>	<code>out = self.data ** exp self.grad += exp * self.data**(exp-1) * out.grad</code>

relu()	$\text{out} = \max(0, x) \mid \text{self.grad} += (\text{out.data} > 0) * \text{out.grad}$
---------------	--

sigmoid()	$s = 1/(1+e^{-x}) \mid \text{self.grad} += s*(1-s) * \text{out.grad}$
------------------	---

tanh()	$t = \tanh(x) \mid \text{self.grad} += (1 - t^2) * \text{out.grad}$
---------------	---

__truediv__	Implemented as $\text{self} * \text{other}^{**}(-1)$
--------------------	--

__neg__	Implemented as $\text{self} * (-1)$
----------------	-------------------------------------

__radd__, __rmul__	Reverse operand versions; allows 2 + val operations
---------------------------	---

2.4 backward() Method

```
def backward(self):
    topo = []
    visited = set()
    def build_topo(v):
        if v not in visited:
            visited.add(v)
            for child in v._prev:
                build_topo(child)
            topo.append(v)
    build_topo(self)
    self.grad = 1.0 # seed: d(loss)/d(loss) = 1
    for node in reversed(topo):
        node._backward()
```

■ Lesson 3: Keras Sequential API (Fashion-MNIST)

Introduction to TensorFlow/Keras. Built classification models using the Sequential API on Fashion-MNIST (28x28 grayscale images, 10 clothing categories).

3.1 Sequential API Workflow

```
from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, Flatten, Dropout

model = Sequential([
    Flatten(input_shape=(28, 28)),    # 2D → 1D: 784 features
    Dense(256, activation='relu'),    # Hidden layer 1
    Dropout(0.3),                    # Regularization
    Dense(128, activation='relu'),    # Hidden layer 2
    Dense(10, activation='softmax')   # Output: 10 classes
])
```

3.2 Key Methods & Concepts

Function / Method	Explanation
Flatten(input_shape=(28,28))	Reshapes 28×28 image to 784-element 1D vector. Required before Dense layers.
Dense(units, activation)	Fully-connected layer. Common activations: 'relu', 'sigmoid', 'softmax', 'tanh'.
Dropout(rate)	Randomly zeros out fraction 'rate' of activations during training. Prevents overfitting.
model.compile(optimizer, loss, metrics)	Configures training. optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy']
sparse_categorical_crossentropy	Use when labels are integers (e.g. 0-9). vs categorical_crossentropy (one-hot labels).
model.fit(X, y, epochs, validation_split)	Trains the model. Returns history object with

loss/accuracy per epoch.

model.evaluate(X_test, y_test)

Returns [loss, accuracy] on test data.

model.predict(X)

Returns raw probability arrays (shape: [n_samples, n_classes]).

model.save('model.keras')

Saves full model (architecture + weights) to disk.

tf.keras.models.load_model('model.keras')

Loads saved model back into memory.

3.3 Data Normalization

```
# Normalize pixel values from [0, 255] to [0.0, 1.0]
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
```

■ Lesson 4: Keras Functional & Subclassing APIs (Fashion-MNIST)

Advanced Keras model building styles: Functional API (non-linear graph architectures) and Model Subclassing (maximum flexibility with custom logic in call()).

4.1 Functional API

```
from tensorflow.keras.layers import Input, Dense, Dropout, Flatten
from tensorflow.keras.models import Model

inputs = Input(shape=(28, 28))          # Define input tensor
x = Flatten()(inputs)                   # Call layer as function
x = Dense(256, activation='relu')(x)
x = Dropout(0.3)(x)
outputs = Dense(10, activation='softmax')(x)
```

```
model = Model(inputs=inputs, outputs=outputs) # Wire graph into Model
```

- **Key difference from Sequential:** Functional API allows branches, multiple inputs/outputs, skip connections. Sequential is purely linear.
- **Shared layers:** Same layer object can be called on different tensors, sharing weights.

4.2 Model Subclassing API

```
from tensorflow.keras import Model

from tensorflow.keras.layers import Dense, Flatten, Dropout

class FashionClassifier(Model):
    def __init__(self):
        super().__init__()
        self.flatten = Flatten()
        self.dense1 = Dense(256, activation='relu')
        self.dropout = Dropout(0.3)
        self.dense2 = Dense(10, activation='softmax')

    def call(self, inputs, training=False):
        x = self.flatten(inputs)
        x = self.dense1(x)
        x = self.dropout(x, training=training) # training flag controls dropout
        return self.dense2(x)
```

- **training=False in call():** During inference, Dropout is disabled. Call `model(x, training=False)` or `model.predict(x)`.
- **model.save() caveat:** Subclassed models need `custom_objects={'FashionClassifier': FashionClassifier}` when loading.
- **np.expand_dims(img, axis=0):** Adds batch dimension: shape (28,28) $\rightarrow$ (1,28,28) for single-image prediction.

■ Lesson 5: Pure PyTorch Autograd & Manual Training Loop

Used PyTorch tensors with `requires_grad=True` to build a 2-layer MLP for California Housing regression WITHOUT using `nn.Module`. Wrote the entire optimization loop by hand.

5.1 Manual Tensor Parameters

```
W1 = torch.randn(8, 32, requires_grad=True) # Trainable weight
b1 = torch.zeros(32, requires_grad=True)    # Trainable bias
W2 = torch.randn(32, 1, requires_grad=True)
b2 = torch.zeros(1, requires_grad=True)
```

5.2 Manual Forward & Backward

```
def model(x):
    hidden = torch.relu(x @ W1 + b1) # Matrix multiply + activation
    output = hidden @ W2 + b2        # Output layer (linear for regression)
    return output

pred = model(X_batch)
loss = ((pred - y_batch) ** 2).mean() # MSE Loss (manual)
loss.backward()                      # Compute gradients

with torch.no_grad():                # Disable gradient tracking for update
    W1 -= learning_rate * W1.grad
    b1 -= learning_rate * b1.grad
W1.grad.zero_()                     # Zero gradients BEFORE next iteration!
```

5.3 Key PyTorch Concepts

Concept / Function	Explanation
--------------------	-------------

requires_grad=True	Tells PyTorch to track this tensor in the computational graph for backprop.
loss.backward()	Computes gradients for all tensors with requires_grad=True in the computation graph.
with torch.no_grad():	Context manager that disables gradient tracking. Mandatory during weight updates & evaluation.
tensor.grad	Stores accumulated gradient after backward(). Starts as None.
tensor.grad.zero_()	Resets gradient to zero. Must do this every iteration! (trailing _ = in-place operation)
torch.randperm(n)	Returns a random permutation of integers 0..n-1. Used for mini-batch shuffling.
tensor.view(-1, 1)	Reshapes tensor. -1 means 'infer this dimension'. Like numpy reshape.
.item()	Extracts a Python scalar from a 0-dim tensor (e.g., loss.item()).

5.4 Mini-Batch Training Pattern

```

for i in range(0, num_samples, batch_size):
    X_batch = X_train[i:i+batch_size]
    y_batch = y_train[i:i+batch_size]
    pred = model(X_batch)
    loss = ((pred - y_batch)**2).mean()
    loss.backward()
    with torch.no_grad():
        W1 -= lr * W1.grad
    W1.grad.zero_()

```

5.5 Regression Metrics

Metric	Formula
MSE (Mean Squared Error)	$\text{mean}((y_{\text{pred}} - y_{\text{true}})^2)$ — penalizes large errors heavily
MAE (Mean Absolute Error)	$\text{mean}(y_{\text{pred}} - y_{\text{true}})$ — robust to outliers
R² Score	$1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$ — 1.0 = perfect fit, 0 = predicts mean, negative = worse than mean

■ Lesson 6: PyTorch nn.Module, Custom Layers, DataLoader (Breast Cancer)

Introduced nn.Module for building modular networks, created a custom linear layer, and used TensorDataset + DataLoader for clean batch loading. Binary classification on Breast Cancer dataset.

6.1 Custom nn.Module Layer

```
class CustomLinear(nn.Module):
    def __init__(self, in_features, out_features):
        super().__init__() # Must call super().__init__()
        self.weight = nn.Parameter( # nn.Parameter = trainable tensor
            torch.randn(out_features, in_features) * 0.01
        )
        self.bias = nn.Parameter(torch.zeros(out_features))

    def forward(self, x): # Must override forward()
        return x @ self.weight.t() + self.bias # .t() = transpose
```

6.2 nn.Module Building Blocks

Component	Explanation
nn.Parameter(tensor)	Wraps a tensor as a trainable parameter. Automatically registered in model.parameters().
nn.Sequential(*layers)	Chains layers sequentially. forward() calls each layer in order.
nn.Linear(in, out)	Built-in fully-connected layer. Equivalent to CustomLinear but optimized.
nn.ReLU()	Activation layer version of F.relu(). Can be placed in nn.Sequential.
nn.CrossEntropyLoss()	Combines LogSoftmax + NLLLoss. Input: raw logits. Target: integer class labels.
model.parameters()	Returns iterator over all trainable parameters. Pass to optimizer.
model.train()	Sets model to training mode (enables dropout, batch norm in train behavior).
model.eval()	Sets model to evaluation mode (disables dropout, fixes batch norm stats).

6.3 Dataset & DataLoader

```

from torch.utils.data import TensorDataset, DataLoader

train_set = TensorDataset(X_train, y_train) # Wraps tensors into a dataset
train_loader = DataLoader(
    train_set,
    batch_size=32,
    shuffle=True      # Shuffle each epoch
)

```

```
# Iterate batches:
for X_batch, y_batch in train_loader:
    ...
```

6.4 Adam Optimizer

```
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001          # Learning rate; Adam default is good starting point
)

# Standard training step:
optimizer.zero_grad() # Clear old gradients
outputs = model(X_batch)
loss = criterion(outputs, y_batch)
loss.backward()       # Compute new gradients
optimizer.step()      # Update weights
```

6.5 Evaluation Pattern

```
model.eval()
with torch.no_grad():
    for X_batch, y_batch in test_loader:
        outputs = model(X_batch)
        predictions = outputs.argmax(dim=1) # Class with highest score
        correct += (predictions == y_batch).sum().item()
        total += y_batch.size(0)
accuracy = correct / total
```

■ Lesson 7: Convolutional Neural Networks (Oxford 102 Flowers)

Built a CNN for multi-class image classification on the Oxford 102 Flowers dataset (8,189 images, 102 classes). Used Torchvision transforms for image preprocessing and a custom Dataset class to load MATLAB-format metadata.

7.1 Convolutional Layer Concepts

Term	Explanation
<code>nn.Conv2d(in_ch, out_ch, kernel_size, padding)</code>	Applies 2D convolutional filters over image. Detects edges, textures, patterns.
<code>kernel_size</code>	Filter size (e.g. 3 means 3×3 filter). Larger = wider receptive field.
<code>padding=1</code>	Adds 1 pixel border of zeros. With 3×3 kernel, preserves spatial dimensions.
<code>nn.MaxPool2d(kernel_size, stride)</code>	Downsamples by taking max value in each window. Reduces spatial size by factor of stride.
<code>stride</code>	How many pixels the filter moves each step. stride=2 halves the spatial dimensions.
Feature Maps	Output channels from Conv2d. Each filter learns a different spatial pattern.
Receptive Field	Region of input pixels that influence a single output neuron. Grows with depth.

7.2 Torchvision Transforms

```
from torchvision import transforms
```

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),      # Resize to fixed size
```

```

transforms.ToTensor(),          # PIL Image → [0,1] FloatTensor (C,H,W)
transforms.Normalize(           # Normalize with ImageNet stats
    mean=[0.485, 0.456, 0.406],
    std=[0.229, 0.224, 0.225]
)
])

```

7.3 Custom Image Dataset Class

```

class FlowerDataset(Dataset):
    def __init__(self, image_paths, labels, transform=None):
        self.image_paths = image_paths
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        image = Image.open(self.image_paths[idx]).convert('RGB') # PIL Image
        label = self.labels[idx]
        if self.transform:
            image = self.transform(image)
        return image, label

```

7.4 CNN Architecture

```

class FlowerCNN(nn.Module):
    def __init__(self, num_classes=102):
        super().__init__()

```

```

self.features = nn.Sequential(
    nn.Conv2d(3, 32, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2,2),
    nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2,2),
    nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2,2),
)

self.classifier = nn.Sequential(
    nn.Flatten(),                # Flatten spatial dims to 1D
    nn.Linear(128*28*28, 512), nn.ReLU(), nn.Dropout(0.5),
    nn.Linear(512, num_classes)
)

def forward(self, x):
    x = self.features(x)
    return self.classifier(x)

```

7.5 MATLAB Dataset Loading

```

import scipy.io as sio

labels = sio.loadmat('imagelabels.mat') # Read .mat file
setid = sio.loadmat('setid.mat')
train_ids = setid['trnid'].flatten()    # Flatten MATLAB array to 1D

```

■ Lesson 8: Custom Dropout & Regularization (Breast Cancer)

Implemented Dropout from scratch (Bernoulli mask), compared it with nn.Dropout, and studied its effect on overfitting. Three models compared: no dropout, custom dropout, PyTorch dropout.

8.1 Inverted Dropout Implementation

```

class CustomDropout(nn.Module):

```

```

def __init__(self, p=0.5):
    super().__init__()
    self.p = p

def forward(self, x):
    if not self.training:          # Disabled during evaluation
        return x

    mask = (torch.rand_like(x) > self.p) # Bernoulli mask (True = keep)
    return x * mask.float() / (1 - self.p) # Scale up to preserve expected value

```

8.2 Key Dropout Concepts

Concept	Explanation
Bernoulli Mask	Each neuron independently set to 0 with probability p . $\text{mask} \sim \text{Bernoulli}(1-p)$.
Inverted Dropout Scaling	During training, active units scaled by $1/(1-p)$. At test time, no modification needed.
self.training flag	nn.Module attribute set by <code>model.train()</code> / <code>model.eval()</code> . Controls dropout behavior.
torch.rand_like(x)	Creates random tensor with same shape/dtype as x , values in $[0,1)$.
Co-adaptation	Neurons relying on specific others; dropout forces independent feature learning.
p=0.5	Common default. For input layers use $p=0.1-0.2$. For hidden layers $0.3-0.5$.

8.3 nn.Dropout vs Custom

PyTorch built-in (equivalent result to custom implementation):

```

nn.Dropout(p=0.5)

# Placement in model:
nn.Sequential(
    nn.Linear(30, 64),
    nn.ReLU(),
    nn.Dropout(0.3),  # After activation
    nn.Linear(64, 32)
)

```

■ Lesson 9: Statistical N-Gram Language Models (Names Dataset)

Built Bigram and Trigram character-level language models using statistical frequency counting. Generated new names by sampling from learned probability distributions.

9.1 Bigram Model

```

from collections import Counter

# Build count matrix: counts[a][b] = # times b follows a
bigram_counts = defaultdict(Counter)

for name in names:
    name = '.' + name + '.'  # Special start/end token
    for a, b in zip(name, name[1:]):
        bigram_counts[a][b] += 1

# Convert counts to probabilities
bigram_probs = {}

for ch, counts in bigram_counts.items():
    total = sum(counts.values())

```

```
bigram_probs[ch] = {k: v/total for k, v in counts.items()}
```

9.2 Name Generation via Sampling

```
import random

def generate_bigram_name(probs):
    name = ""
    ch = '.' # Start token
    while True:
        next_chars = list(probs[ch].keys())
        weights = list(probs[ch].values())
        ch = random.choices(next_chars, weights=weights)[0] # Weighted sampling
        if ch == '.':
            break
        name += ch
    return name
```

9.3 PyTorch Tensor Version

```
# Build count matrix as tensor
N = torch.zeros((vocab_size, vocab_size), dtype=torch.int32)

for name in names:
    for a, b in zip(name, name[1:]):
        N[stoi[a], stoi[b]] += 1

# Convert to probability matrix (row normalization)
P = N.float()
P /= P.sum(dim=1, keepdim=True) # Each row sums to 1
```

```
# Sample next character
ix = torch.multinomial(P[current_ix], num_samples=1).item()
```

9.4 Key Functions

Function	Explanation
torch.multinomial(probs, num_samples=1)	Samples indices from a probability distribution. Returns index tensor.
random.choices(population, weights)	Python stdlib: weighted random sampling from a list.
P.sum(dim=1, keepdim=True)	Row-wise sum; keepdim=True preserves shape for broadcasting.
stoi / itos	String-to-index / index-to-string vocabulary dictionaries.
Trigram	Conditions on last 2 characters: $P(c \mid a, b)$. Needs 3D count tensor $N[a][b][c]$.

■ Lesson 10: Bengio MLP — Neural N-Gram Language Model

Replaced statistical bigrams with a neural MLP (Bengio et al. 2003). Characters are embedded into dense vectors, a fixed-size context window is fed through hidden layers to predict the next character.

10.1 Character Embedding Lookup

```
# Vocabulary
chars = sorted(set("".join(words)))
stoi = {s: i+1 for i, s in enumerate(chars)}
stoi['.'] = 0 # Special padding/start/end token
```

```

itos = {i: s for s, i in stoi.items()}
vocab_size = len(stoi) # e.g. 27 for a-z + '.'

# Embedding: maps each character index to a dense vector
nn.Embedding(vocab_size, embedding_dim)

# Access: embedding(torch.tensor([3])) → vector of size embedding_dim

```

10.2 Dataset Construction

```

block_size = 3 # Context window: predict 4th char given 3 previous
X, Y = [], []
for word in words:
    context = [0] * block_size # Pad with '.' token
    for ch in word + '.':
        ix = stoi[ch]
        X.append(context)
        Y.append(ix)
        context = context[1:] + [ix] # Slide window right
X = torch.tensor(X) # Shape: (N, block_size)
Y = torch.tensor(Y) # Shape: (N,)

```

10.3 NGram MLP Architecture

```

class NGramMLP(nn.Module):
    def __init__(self, vocab_size, block_size, embedding_dim, hidden_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        # Input size = block_size × embedding_dim (concatenated context)
        self.fc1 = nn.Linear(block_size * embedding_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, vocab_size)

```

```
def forward(self, x):
    x = self.embedding(x)      # (B, block_size) → (B, block_size, emb_dim)
    x = x.view(x.size(0), -1)  # Flatten: (B, block_size*emb_dim)
    x = F.relu(self.fc1(x))
    return self.fc2(x)         # Logits over vocab
```

10.4 torch.save / torch.load Checkpoint

```
# Save checkpoint (includes model state AND optimizer state)
torch.save({
    'epoch': epoch,
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'validation_loss': val_loss,
}, 'checkpoint.pth')

# Load checkpoint
checkpoint = torch.load('checkpoint.pth', map_location=device)
model.load_state_dict(checkpoint['model_state_dict'])
optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
```

■ Lesson 11: Batch Normalization & Embedding Visualization (PCA)

Added Batch Normalization to the MLP to stabilize training. Visualized character embeddings using PCA to understand what the embedding table learns.

11.1 Batch Normalization

```
nn.BatchNorm1d(num_features) # For 1D/2D data (FC layers)
```

```
nn.BatchNorm2d(num_features) # For 4D data (Conv layers)

# In forward pass:
x = F.relu(self.fc1(x))
x = self.bn1(x)           # Normalize AFTER activation (common convention)
```

Concept	Explanation
What it does	Normalizes mini-batch activations to zero mean and unit variance. Then applies learnable γ (scale) and β (shift).
γ and β	Learnable parameters (nn.Parameter). γ starts at 1, β starts at 0. Allows model to un-normalize if needed.
Training vs Eval	During training: uses batch mean/variance. During eval: uses running mean/variance (tracked across batches).
Benefits	Reduces internal covariate shift, allows higher learning rates, reduces sensitivity to initialization.
Placement	Typically: Linear $\rightarrow$ BN $\rightarrow$ Activation. Sometimes after activation.

11.2 PCA Embedding Visualization

```
from sklearn.decomposition import PCA

# Extract learned embedding weights
embeddings = model.embedding.weight.detach().numpy() # Shape: (vocab_size, emb_dim)

# Reduce to 2D
```

```

pca = PCA(n_components=2)
reduced = pca.fit_transform(embeddings) # Shape: (vocab_size, 2)

# Plot
for i in range(vocab_size):
    plt.scatter(reduced[i, 0], reduced[i, 1])
    plt.text(reduced[i, 0], reduced[i, 1], itos[i], fontsize=12)

```

- **What PCA reveals:** Characters with similar context distributions (e.g. vowels: a,e,i,o,u) cluster together in embedding space.

■ Lesson 12: Deep MLPs with Residual Connections & Experiments

Extended the NGram MLP with multiple hidden layers, Batch Normalization, and hyperparameter experimentation (embedding size, hidden size, number of layers).

12.1 Configurable Multi-Layer MLP

```

class NGramMLP(nn.Module):
    def __init__(self, vocab_size, block_size, embedding_dim, hidden_dim, num_hidden_layers):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        layers = [nn.Linear(block_size*embedding_dim, hidden_dim), nn.ReLU()]
        for _ in range(num_hidden_layers - 1):
            layers += [nn.Linear(hidden_dim, hidden_dim), nn.ReLU()]
        layers.append(nn.Linear(hidden_dim, vocab_size))
        self.network = nn.Sequential(*layers)

    def forward(self, x):
        x = self.embedding(x).view(x.size(0), -1)
        return self.network(x)

```

12.2 Hyperparameter Experimentation

```
experiments = [  
    (embedding_dim=10, hidden_dim=100, num_layers=1),  
    (embedding_dim=20, hidden_dim=200, num_layers=2),  
    (embedding_dim=30, hidden_dim=300, num_layers=2),  
]  
  
# Run each, record validation loss, find best config
```

- **torch.randperm(n):** Generates random permutation for shuffling training data each epoch.

■ Lesson 13: Character-Level Language Model on TinyStories

Moved from character vocabulary to the full TinyStories dataset. Used character-level tokenization and trained a larger MLP to predict next characters in children's stories.

13.1 HuggingFace datasets Library

```
from datasets import load_dataset  
  
dataset = load_dataset('roneneldan/TinyStories')  
  
train_data = dataset['train'].select(range(1000)) # Use subset  
valid_data = dataset['validation'].select(range(200))
```

13.2 Building Character Vocabulary from Dataset

```
train_text = ' '.join(train_data[:1000]['text']) # Concatenate all stories  
  
chars = sorted(set(train_text)) # All unique characters  
stoi = {c: i for i, c in enumerate(chars)}  
itos = {i: c for c, i in stoi.items()}  
  
vocab_size = len(stoi)
```

13.3 Character-Level Dataset

```
class CharDataset(Dataset):
    def __init__(self, text, block_size):
        data = [stoi[c] for c in text] # Encode text to integers
        self.data = torch.tensor(data, dtype=torch.long)
        self.block_size = block_size

    def __len__(self):
        return len(self.data) - self.block_size

    def __getitem__(self, idx):
        x = self.data[idx : idx + self.block_size] # Input sequence
        y = self.data[idx+1 : idx + self.block_size + 1] # Target: shifted by 1
        return x, y
```

■ Lesson 14: Autoregressive MLP with Position Embeddings (TinyStories)

Enhanced the language model with a token embedding table AND separate positional embeddings. Trained on next-token prediction (NTP) — the core objective of language models.

14.1 Key Hyperparameters

```
BLOCK_SIZE = 8 # Context length (tokens seen at once)
BATCH_SIZE = 256
EMBEDDING_DIM = 64 # Size of each token embedding vector
HIDDEN_SIZE = 512
DROPOUT = 0.3
LEARNING_RATE = 0.001
```

```
EPOCHS = 10
```

14.2 Positional Embeddings

```
class PositionalEmbeddingMLP(nn.Module):
    def __init__(self, vocab_size, block_size, embed_dim, hidden_size):
        super().__init__()
        self.token_embed = nn.Embedding(vocab_size, embed_dim) # What is it
        self.pos_embed = nn.Embedding(block_size, embed_dim) # Where is it
        ...

    def forward(self, x):
        B, T = x.shape
        tok = self.token_embed(x) # (B, T, embed_dim)
        pos = self.pos_embed(torch.arange(T, device=x.device)) # (T, embed_dim)
        x = tok + pos # Broadcast add
        x = x.view(B, -1) # Flatten
        ...
```

- **Why positional embeddings?:** Without them, the model has no sense of token order. Position 0 and position 7 would be treated identically.
- **torch.arange(T):** Creates tensor [0, 1, 2, ..., T-1] for position indices.

14.3 NTP Loss Reshape Pattern

```
# logits shape: (B, T, vocab_size)
# targets shape: (B, T)
loss = criterion(
    logits.view(-1, vocab_size), # Reshape to (B*T, vocab_size)
    targets.view(-1) # Reshape to (B*T,)
)
```

■ Lesson 15: WaveNet Architecture for Text Generation (TinyStories + BPE)

Replaced MLP with a WaveNet — a stack of 1D causal dilated convolutional blocks. Used GPT-2's BPE tokenizer (tiktoken) instead of character-level encoding.

15.1 BPE Tokenization with tiktoken

```
import tiktoken

enc = tiktoken.get_encoding('gpt2')    # GPT-2 tokenizer: 50,257 vocab

tokens = enc.encode('Hello world')     # List[int]
text  = enc.decode([15496, 995])      # List[int] → string

VOCAB_SIZE = 50257                    # GPT-2 vocab size
enc.eot_token                # End-of-text token ID
```

15.2 Causal Convolution

```
class CausalConv1d(nn.Module):

    def __init__(self, in_ch, out_ch, kernel_size, dilation=1):
        super().__init__()
        self.padding = (kernel_size - 1) * dilation  # Left pad only
        self.conv = nn.Conv1d(
            in_ch, out_ch, kernel_size,
            padding=self.padding, dilation=dilation
        )

    def forward(self, x):
        x = self.conv(x)
        if self.padding > 0:
            x = x[:, :, :-self.padding]  # Remove right-side padding (non-causal)
```

- **Why causal?:** Prevents the model from 'seeing the future'. Position t can only depend on positions $\leq t$.
- **Dilation:** Gaps between filter elements: dilation=2 means the filter looks at positions 0, 2, 4 (skips 1, 3). Exponentially expands receptive field.

15.3 WaveNet Architecture

```
class WaveNet(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_channels, num_blocks):
        super().__init__()

        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.input_projection = nn.Conv1d(embed_dim, hidden_channels, kernel_size=1)

        # Dilations: 1, 2, 4, 8, 16, 32 (exponential growth)
        dilations = [2**i for i in range(num_blocks)]
        self.residual_blocks = nn.ModuleList([
            ResidualBlock(hidden_channels, d) for d in dilations
        ])

        self.output_projection = nn.Conv1d(hidden_channels, hidden_channels, 1)
        self.fc = nn.Linear(hidden_channels, vocab_size)

    def forward(self, x):
        x = self.embedding(x)          # (B, L, embed_dim)
        x = x.transpose(1, 2)          # (B, embed_dim, L) for Conv1d
        x = self.input_projection(x)
        for block in self.residual_blocks:
            x = block(x)
        x = self.output_projection(x)
        x = x.transpose(1, 2)          # Back to (B, L, hidden)
        return self.fc(x)              # (B, L, vocab_size)
```

15.4 nn.ModuleList vs nn.Sequential

Feature	nn.ModuleList vs nn.Sequential
nn.ModuleList([layer1, layer2])	Registers layers as parameters but does NOT define forward pass. You manually loop over them.
nn.Sequential(*layers)	Automatically chains layers: output of one is input to next.
When to use ModuleList	When layers need custom connections (e.g., residual adds, skip connections).

15.5 Text Generation with @torch.no_grad()

```

@torch.no_grad()          # Decorator = automatic no_grad context
def generate(prompt, max_new_tokens=100):
    tokens = enc.encode(prompt)
    for _ in range(max_new_tokens):
        context = tokens[-BLOCK_SIZE:] # Use only last BLOCK_SIZE tokens
        # Pad if needed
        if len(context) < BLOCK_SIZE:
            context = [enc.eot_token] * (BLOCK_SIZE - len(context)) + context
        x = torch.tensor(context).unsqueeze(0).to(device) # Add batch dim
        logits = model(x)
        logits = logits[:, -1, :]      # Only predict from LAST position
        probs = torch.softmax(logits, dim=-1)
        next_token = torch.multinomial(probs, 1).item()
        tokens.append(next_token)
    return enc.decode(tokens)

```

■ Lessons 16–17: 1D CNN for Text Classification (Names → Nationality)

Used 1D Convolutions (nn.Conv1d) to classify names by nationality (18 languages). Learned custom Dataset with variable-length sequences and a custom collate_fn for dynamic padding.

16.1 Text Preprocessing

```
from unidecode import unidecode  # Convert accented chars: Ñ → N

name = unidecode(name.strip().lower())

# Character encoding
def encode(name):
    return [stoi[c] for c in name if c in stoi]

# Label encoding
class_names = sorted(os.listdir(DATA_DIR))
label2id = {name: i for i, name in enumerate(class_names)}
```

16.2 Custom Dataset with Variable-Length Sequences

```
class NamesDataset(Dataset):
    def __init__(self, names, labels):
        self.names = [torch.tensor(encode(n), dtype=torch.long) for n in names]
        self.labels = torch.tensor(labels, dtype=torch.long)

    def __len__(self): return len(self.names)

    def __getitem__(self, idx):
        return self.names[idx], self.labels[idx]
```

16.3 Custom collate_fn — Dynamic Padding

```

def collate_fn(batch):
    names, labels = zip(*batch)    # Unzip list of (name, label) tuples
    max_len = max(n.size(0) for n in names) # Longest name in batch

    # Pad all names to max_len with 0 (padding token)
    padded = torch.zeros(len(names), max_len, dtype=torch.long)
    for i, n in enumerate(names):
        padded[i, :n.size(0)] = n

    return padded, torch.stack(labels)

train_loader = DataLoader(dataset, batch_size=64, collate_fn=collate_fn)

```

16.4 1D CNN Classifier

```

class CNNClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, num_classes, num_filters=128):
        super().__init__()

        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

        # Conv1d: operates on sequence dimension
        self.conv3 = nn.Conv1d(embed_dim, num_filters, kernel_size=3, padding=1)
        self.conv5 = nn.Conv1d(embed_dim, num_filters, kernel_size=5, padding=2)
        self.pool = nn.AdaptiveMaxPool1d(1) # Max over entire sequence
        self.fc = nn.Linear(num_filters * 2, num_classes)

    def forward(self, x):
        x = self.embedding(x) # (B, L, embed_dim)
        x = x.permute(0, 2, 1) # (B, embed_dim, L) for Conv1d
        x3 = F.relu(self.conv3(x))
        x5 = F.relu(self.conv5(x))
        x3 = self.pool(x3).squeeze(-1) # (B, num_filters)

```

```

x5 = self.pool(x5).squeeze(-1)

x = torch.cat([x3, x5], dim=1) # Concatenate multi-scale features

return self.fc(x)

```

Function / Layer	Explanation
nn.Conv1d(in_ch, out_ch, kernel_size)	Applies 1D filters along sequence dimension. in_ch must match embedding_dim after permute.
nn.AdaptiveMaxPool1d(output_size=1)	Reduces sequence to single value by taking max. Makes output independent of input length.
x.permute(0, 2, 1)	Rearranges tensor dimensions. (B,L,C) → (B,C,L) required by Conv1d.
nn.Embedding(..., padding_idx=0)	Embedding for index 0 is always zero-vector and not updated. Used for padding tokens.
torch.cat([a, b], dim=1)	Concatenates tensors along dimension 1 (feature dimension).
ConfusionMatrixDisplay.from_predictions()	Creates confusion matrix plot directly from prediction arrays.

■ Lesson 18: Recurrent Neural Networks (RNN) — Character-Level Name Generation

Implemented nn.RNN for autoregressive name generation. Characters are fed one at a time; hidden state carries memory across the sequence.

18.1 RNN Architecture

```

class CharRNN(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_size, num_layers=1):

```

```

super().__init__()
self.embedding = nn.Embedding(vocab_size, embed_dim)
self.rnn = nn.RNN(
    input_size=embed_dim,
    hidden_size=hidden_size,
    num_layers=num_layers,
    batch_first=True, # (B, T, features) instead of (T, B, features)
    nonlinearity='tanh' # or 'relu'
)
self.fc = nn.Linear(hidden_size, vocab_size)

def forward(self, x, hidden=None):
    x = self.embedding(x) # (B, T, embed_dim)
    out, hidden = self.rnn(x, hidden) # out: (B,T,hidden), hidden: (layers,B,hidden)
    logits = self.fc(out) # (B, T, vocab_size)
    return logits, hidden

```

18.2 RNN Concepts

Concept	Explanation
Hidden State h_t	$h_t = \tanh(W_{hh} * h_{t-1} + W_{xh} * x_t + b)$. Carries information across time steps.
batch_first=True	Changes input shape from (T,B,C) to (B,T,C). More intuitive when T is sequence length.
num_layers	Stacks multiple RNN layers (deep RNN). Output of layer i is input of layer i+1.
Vanishing Gradients	Gradients shrink exponentially during BPTT through long sequences. RNNs suffer from this.
Truncated BPTT	Backpropagating through only a fixed number

of steps to limit memory/gradient issues.

18.3 Name Generation with RNN

```
def generate_name(model, stoi, itos, device, max_len=20):
    model.eval()
    with torch.no_grad():
        x = torch.tensor([[stoi['.']], dtype=torch.long).to(device) # Start token
        hidden = None
        name = ""
        for _ in range(max_len):
            logits, hidden = model(x, hidden) # Pass hidden state forward
            probs = torch.softmax(logits[0, -1], dim=-1)
            ix = torch.multinomial(probs, 1).item()
            if itos[ix] == '.':
                break
            name += itos[ix]
            x = torch.tensor([[ix]], dtype=torch.long).to(device)
        return name
```

■ Lesson 19: LSTM & GRU — Advanced Recurrent Models

Replaced the vanilla RNN with LSTM and GRU to solve the vanishing gradient problem. LSTMs add a Cell State governed by three gates; GRUs simplify this with two gates.

19.1 LSTM Architecture

```
self.lstm = nn.LSTM(
    input_size=embed_dim,
    hidden_size=hidden_size,
    num_layers=num_layers,
```

```

    batch_first=True,
    dropout=0.3    # Dropout between LSTM layers (if num_layers > 1)
)

# LSTM returns: (output, (h_n, c_n))
out, (h_n, c_n) = self.lstm(x, (h_0, c_0))

# h_n = final hidden state
# c_n = final cell state

```

19.2 LSTM Gates

Gate	Formula & Role
Forget Gate (f_t)	$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$ — How much of old cell state to keep
Input Gate (i_t)	$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$ — How much of new candidate to add
Candidate ($\tilde{c}_t$)	$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$ — New candidate cell state values
Cell Update (c_t)	$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ — Updated cell state (long-term memory)
Output Gate (o_t)	$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$ — What part of cell state to output
Hidden State (h_t)	$h_t = o_t \odot \tanh(c_t)$ — Short-term output state

19.3 GRU Architecture

```

self.gru = nn.GRU(
    input_size=embed_dim,

```

```

hidden_size=hidden_size,
num_layers=num_layers,
batch_first=True
)
# GRU returns: (output, h_n)
out, h_n = self.gru(x, h_0)

```

GRU Gate	Formula & Role
Reset Gate (r_t)	$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$ — How much old hidden state to forget
Update Gate (z_t)	$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$ — Balance between old and new state
New Hidden	$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t])$ — Candidate hidden state
Output	$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$ — Weighted mix of old & new

GRU advantages: Fewer parameters than LSTM, faster training, often comparable performance. LSTM advantage: Separate cell & hidden state for better long-term memory.

■ Lesson 20: Hugging Face Pipeline API & Tokenizers

First introduction to the Hugging Face ecosystem. Used the high-level pipeline API for instant inference, then manually loaded tokenizers and models for custom control.

20.1 Pipeline API

```

from transformers import pipeline

# Sentiment Analysis (default model: distilbert-base-uncased-finetuned-sst-2-english)
sentiment = pipeline('sentiment-analysis')

```

```

result = sentiment(['I love this!', 'This is terrible.'])
# → [{'label': 'POSITIVE', 'score': 0.99}, {'label': 'NEGATIVE', 'score': 0.97}]

# Text Generation
generator = pipeline('text-generation', model='distilgpt2')
result = generator('Deep Learning is', max_new_tokens=30, num_return_sequences=1)

# Zero-Shot Classification
classifier = pipeline('zero-shot-classification')
result = classifier(
    'I bought a new laptop.',
    candidate_labels=['technology', 'sports', 'food']
)

```

20.2 Manual Tokenizer + Model

```

from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch

MODEL_NAME = 'distilbert-base-uncased-finetuned-sst-2-english'
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForSequenceClassification.from_pretrained(MODEL_NAME)

text = 'I love deep learning!'
# Tokenize: returns dict with input_ids, attention_mask, etc.
inputs = tokenizer(text, return_tensors='pt', truncation=True, max_length=128)

with torch.no_grad():
    outputs = model(**inputs) # Pass dict as keyword args with **
    logits = outputs.logits # Raw scores
    probs = torch.softmax(logits, -1) # Convert to probabilities

```

```
pred_id = probs.argmax().item()
```

```
label = model.config.id2label[pred_id] # Map index → class name
```

20.3 Tokenizer Outputs Explained

Output Key	Explanation
input_ids	Tensor of token IDs (integers). [CLS] id at start, [SEP] id at end for BERT.
attention_mask	1 for real tokens, 0 for padding tokens. Tells model what to attend to.
token_type_ids	Differentiates sentence A from sentence B (for sentence-pair tasks). 0=A, 1=B.
return_tensors='pt'	Return PyTorch tensors. Options: 'pt' (PyTorch), 'tf' (TensorFlow), 'np' (NumPy).

■ Lesson 21: BERT Fine-Tuning Introduction (Emotion Classification)

First hands-on fine-tuning of a pre-trained BERT model for multi-class emotion classification on the SMILE Twitter dataset (5 classes: happy, sad, angry, surprise, disgust).

21.1 Data Preparation

```
df = pd.read_csv(DATA_PATH, names=['id', 'text', 'label'], header=None)
df = df[['text', 'label']].dropna()

# Filter to valid classes only
VALID_EMOTIONS = ['happy', 'angry', 'sad', 'surprise', 'disgust']
df = df[df['label'].isin(VALID_EMOTIONS)].copy()
```

```

# Build label encodings
labels = sorted(df['label'].unique())
label2id = {l: i for i, l in enumerate(labels)}
id2label = {i: l for l, i in label2id.items()}
df['label'] = df['label'].map(label2id)

# Stratified split
train_df, val_df = train_test_split(df, test_size=0.2, stratify=df['label'])

```

21.2 Custom BERT Dataset

```

class EmotionDataset(Dataset):
    def __init__(self, dataframe, tokenizer, max_length):
        self.texts = dataframe['text'].tolist()
        self.labels = dataframe['label'].tolist()
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self): return len(self.texts)

    def __getitem__(self, idx):
        encoding = self.tokenizer(
            self.texts[idx],
            truncation=True,
            max_length=self.max_length
            # NOTE: No padding here — DataCollatorWithPadding handles it per batch
        )
        return {
            'input_ids': encoding['input_ids'],
            'attention_mask': encoding['attention_mask'],

```

```

        'labels':      self.labels[idx]
    }

```

■ Lesson 22: RNN for Nationality Classification (Sequence-to-One)

Applied CharRNN for sequence-to-one classification (predict nationality from name). Key new concept: using only the final hidden state as the class representation.

22.1 Sequence-to-One vs Sequence-to-Sequence

Task Type	Description
Sequence-to-One	Process full sequence with RNN, use ONLY the LAST hidden state for classification. (Name → Nationality)
Sequence-to-Sequence	Use ALL time step outputs. (Name generation, translation)
out[:, -1, :]	Select last time step from RNN output tensor of shape (B, T, hidden_size).
h_n[-1]	Final layer, final time step hidden state from nn.RNN/LSTM output.

22.2 CharacterRNN for Classification

```

class CharacterRNN(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_size, num_classes, num_layers=1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.rnn = nn.RNN(embed_dim, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, num_classes)

```

```

def forward(self, x):
    emb = self.embedding(x)          # (B, T, embed_dim)
    out, h_n = self.rnn(emb)         # out: (B,T,hidden), h_n: (layers,B,hidden)
    last = out[:, -1, :]             # Last time step: (B, hidden)
    return self.fc(last)             # (B, num_classes)

```

■ Lesson 23: Hugging Face Full Manual Inference Pipeline

Deep dive into the 5-step manual inference process: Load tokenizer → Preprocess text → Forward pass → Get logits → Post-process output. Demystifies what pipeline() does internally.

23.1 The 5 Steps of Manual Inference

```

### Step 1: Load tokenizer & model

tokenizer = AutoTokenizer.from_pretrained('distilbert-base-uncased-finetuned-sst-2-english')
model     = AutoModelForSequenceClassification.from_pretrained('...')


### Step 2: Preprocess (tokenize)

inputs = tokenizer(
    ['I love ML!', 'This is bad.'],
    return_tensors='pt',    # PyTorch tensors
    truncation=True,
    padding=True,          # Pad to longest in batch
    max_length=512
)


### Step 3: Forward pass

with torch.no_grad():
    outputs = model(**inputs)

```

```

### Step 4: Get logits
logits = outputs.logits    # Shape: (batch_size, num_classes)

### Step 5: Post-process
probs = torch.softmax(logits, dim=-1)
pred_ids = probs.argmax(dim=-1).tolist()
labels = [model.config.id2label[i] for i in pred_ids]

```

23.2 Key Transformers Classes

Class / Function	Explanation
AutoTokenizer.from_pretrained(name)	Loads correct tokenizer for any model. Auto-detects type (BPE, WordPiece, SentencePiece).
AutoModelForSequenceClassification	Loads BERT/DistilBERT/RoBERTa + classification head for text classification.
outputs.logits	Raw unnormalized scores. Shape (batch, num_labels). Apply softmax for probabilities.
model.config.id2label	Dict mapping integer class index → class name string.
tokenizer.convert_tokens_to_ids()	Convert token strings to their IDs in the vocabulary.
tokenizer.decode(ids)	Convert list of token IDs back to human-readable string.

■ Lesson 25: BERT Fine-Tuning — Complete Training Pipeline & HF Hub Upload

Complete professional fine-tuning pipeline: custom Dataset, DataCollatorWithPadding, AdamW optimizer with linear warmup scheduler, gradient clipping, best-model checkpointing, and Hugging Face Hub upload.

25.1 DataCollatorWithPadding

```
from transformers import DataCollatorWithPadding

data_collator = DataCollatorWithPadding(tokenizer=tokenizer)

# Pass to DataLoader:
train_loader = DataLoader(
    train_dataset,
    batch_size=16,
    shuffle=True,
    collate_fn=data_collator  # Pads each batch to its own max length
)
```

- **Why use this?:** Instead of padding everything to the global max, each batch is padded to the max within that batch. Much more memory-efficient.

25.2 AutoModelForSequenceClassification with Labels

```
model = AutoModelForSequenceClassification.from_pretrained(
    'bert-base-uncased',
    num_labels=5,
    label2id=label2id,  # {'happy': 0, 'angry': 1, ...}
    id2label=id2label  # {0: 'happy', 1: 'angry', ...}
)

# When you pass labels= to model(), it computes loss automatically:
outputs = model(input_ids=ids, attention_mask=mask, labels=labels)
loss = outputs.loss  # Cross-entropy loss (computed internally)
logits = outputs.logits  # Shape: (batch, num_labels)
```

25.3 AdamW Optimizer

```
optimizer = torch.optim.AdamW(  
    model.parameters(),  
    lr=2e-5,          # Typical for fine-tuning: 1e-5 to 5e-5  
    weight_decay=0.01 # L2 regularization on weights (not biases)  
)
```

- **AdamW vs Adam:** AdamW correctly decouples weight decay from gradient updates (Adam incorrectly applied decay to adaptive LR). Better generalization for transformers.

25.4 Linear Warmup Scheduler

```
from transformers import get_linear_schedule_with_warmup  
  
total_steps = len(train_loader) * NUM_EPOCHS  
warmup_steps = int(total_steps * 0.1) # 10% warmup  
  
scheduler = get_linear_schedule_with_warmup(  
    optimizer,  
    num_warmup_steps=warmup_steps,  
    num_training_steps=total_steps  
)  
  
# Call after optimizer.step():  
optimizer.step()  
scheduler.step()
```

- **What warmup does:** LR starts near 0, linearly increases to max_lr over warmup_steps, then linearly decays to 0. Prevents early unstable updates that could destroy pre-trained weights.

25.5 Gradient Clipping

```
torch.nn.utils.clip_grad_norm_(
    model.parameters(),
    max_norm=1.0      # Clip gradient norm to this value
)
# Call BEFORE optimizer.step(), AFTER loss.backward()
```

- **Why clip?:** Transformers can have exploding gradients. Clipping ensures the gradient vector's L2 norm never exceeds `max_norm`, stabilizing training.

25.6 Full Training Step

```
for batch in train_loader:
    input_ids   = batch['input_ids'].to(device)
    attention_mask = batch['attention_mask'].to(device)
    labels      = batch['labels'].to(device)

    optimizer.zero_grad()

    outputs = model(input_ids=input_ids, attention_mask=attention_mask, labels=labels)
    loss    = outputs.loss
    logits  = outputs.logits

    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    optimizer.step()
    scheduler.step()
```

25.7 Model Checkpointing

```
best_f1 = 0.0
```

```

if val_f1 > best_f1:
    best_f1 = val_f1
    os.makedirs(OUTPUT_DIR, exist_ok=True)
    model.save_pretrained(OUTPUT_DIR)    # Saves config.json + pytorch_model.bin
    tokenizer.save_pretrained(OUTPUT_DIR) # Saves tokenizer_config.json + vocab files

# Reload best model:
model = AutoModelForSequenceClassification.from_pretrained(OUTPUT_DIR)

```

25.8 Hugging Face Hub Upload

```

from huggingface_hub import login

login() # Prompts for token OR: login(token='hf_...')

model.push_to_hub('username/repo-name')
tokenizer.push_to_hub('username/repo-name')
# → Uploads to https://huggingface.co/username/repo-name

```

■ Master Quick Reference: All Important Functions & Methods

PyTorch Core

Function / Method	Usage
<code>torch.tensor(data, dtype=torch.float32)</code>	Create tensor from Python list/numpy array
<code>torch.zeros(shape)</code>	All-zeros tensor

torch.randn(shape)	Random normal tensor (mean=0, std=1)
torch.randperm(n)	Random permutation [0..n-1]
torch.arange(n)	Integer sequence [0, 1, ..., n-1]
torch.ones_like(t) / zeros_like(t)	Same shape/dtype as t, filled with 1s or 0s
tensor.view(shape) / .reshape(shape)	Reshape without copying (view requires contiguous memory)
tensor.permute(dims)	Reorder all dimensions: (B,T,C) $\rightarrow$ (B,C,T) with .permute(0,2,1)
tensor.squeeze(dim)	Remove size-1 dimensions. squeeze(-1) removes last dim if size==1.
tensor.unsqueeze(dim)	Add size-1 dimension: (B,) $\rightarrow$ (B,1) with unsqueeze(-1)
tensor.to(device)	Move tensor to CPU/GPU/MPS device
tensor.detach().cpu().numpy()	Convert gradient-tracked tensor to numpy (detach breaks graph)
tensor.size(0) / tensor.shape[0]	Get dimension size (batch size = dim 0)
torch.cat([a, b], dim=1)	Concatenate tensors along a dimension
torch.stack([a, b], dim=0)	Stack tensors into a new dimension
torch.argmax(t, dim=-1)	Index of maximum value along last dimension
torch.softmax(t, dim=-1)	Convert logits to probabilities summing to 1
torch.multinomial(probs, n)	Sample n indices according to probability distribution
torch.no_grad() / @torch.no_grad()	Disable gradient computation (saves memory at inference)

model.state_dict()	OrderedDict of all parameter tensors
model.load_state_dict(sd)	Load saved parameters back into model
torch.save(obj, path)	Save any Python object (dict, tensor) to disk
torch.load(path, map_location)	Load saved object. map_location='cpu' to load GPU model on CPU

PyTorch nn Module Functions

Function / Method	Usage
nn.Linear(in, out)	Fully-connected layer: $y = xW^T + b$
nn.Embedding(vocab_size, dim, padding_idx)	Lookup table: int index $\rightarrow$ dense vector
nn.Conv1d(in_ch, out_ch, kernel_size)	1D convolution for sequences
nn.Conv2d(in_ch, out_ch, kernel_size, padding)	2D convolution for images
nn.MaxPool1d/MaxPool2d(kernel_size)	Max pooling: reduces spatial dimensions
nn.AdaptiveMaxPool1d(output_size)	Max pool to fixed output size regardless of input size
nn.ReLU() / F.relu(x)	Rectified Linear Unit activation
nn.Tanh() / F.tanh(x)	Hyperbolic tangent activation
nn.Sigmoid() / F.sigmoid(x)	Sigmoid activation: output in (0,1)
F.softmax(x, dim=-1)	Softmax over last dimension
nn.Dropout(p)	Inverted dropout with probability p
nn.BatchNorm1d/2d(num_features)	Batch normalization layer

nn.RNN/LSTM/GRU(input_size, hidden_size)	Recurrent layers
nn.ModuleList(list_of_modules)	List of modules; each registered as parameter
nn.Sequential(*modules)	Chain of modules called in order
nn.CrossEntropyLoss()	Loss for multi-class: input=logits, target=integers
nn.BCEWithLogitsLoss()	Binary cross-entropy with logits
nn.MSELoss()	Mean squared error loss for regression

Optimizers

Optimizer	Best Use Case
torch.optim.SGD(params, lr, momentum)	Simple gradient descent with optional momentum. Good baseline.
torch.optim.Adam(params, lr=1e-3)	Adaptive learning rates. Default choice for most tasks.
torch.optim.AdamW(params, lr, weight_decay)	Adam with correct weight decay. Preferred for transformer fine-tuning.
optimizer.zero_grad()	Must call before each backward pass to reset accumulated gradients.
optimizer.step()	Updates all parameters using their computed gradients.

Hugging Face Key Classes

Class / Function	Usage
AutoTokenizer.from_pretrained(name)	Load tokenizer auto-matching model architecture

AutoModelForSequenceClassification	Load pre-trained model + classification head
DataCollatorWithPadding(tokenizer)	Dynamic per-batch padding collator
get_linear_schedule_with_warmup()	LR warmup + linear decay scheduler
pipeline(task, model)	High-level instant inference: 'sentiment-analysis', 'text-generation', 'zero-shot-classification'
model.save_pretrained(dir)	Save model weights + config to directory
tokenizer.save_pretrained(dir)	Save tokenizer files to directory
model.push_to_hub(repo_name)	Upload model to Hugging Face Hub
tokenizer.push_to_hub(repo_name)	Upload tokenizer to Hugging Face Hub
login()	Authenticate with Hugging Face Hub (token prompt)

Evaluation Metrics Summary

Metric	Formula & When to Use
Accuracy	Correct / Total. Simple but misleading for imbalanced datasets.
Precision	$TP / (TP + FP)$. 'Of all I predicted positive, how many were?'
Recall	$TP / (TP + FN)$. 'Of all actual positives, how many did I catch?'
F1-Score	$2 * (P * R) / (P + R)$. Harmonic mean of precision & recall. Good for imbalanced.
Weighted F1	Average F1 weighted by class support. Best for multi-class with imbalance.

MSE

$(y_{\text{pred}} - y_{\text{true}})^2$. Penalizes large errors heavily.
Unit = target²

MAE

$|y_{\text{pred}} - y_{\text{true}}|$. More robust to outliers than MSE.

R² Score

Proportion of variance explained. 1.0 = perfect,
0 = no better than mean.

Perplexity

$\exp(\text{cross-entropy loss})$. Used for language models. Lower = better. Perfect = 1.